02-running-specs/06/spec/coffee_spec.rb
```
class Coffee
  def ingredients
    @ingredients ||= []
  end

  def add(ingredient)
    ingredients << ingredient
  end

  def price
    1.00 + ingredients.size * 0.25
  end
end
```

With your implementation in place, rerun RSpec with the --only-failures option:

```
$ rspec --only-failures
Run options: include {:last_run_status=>"failed"}
.

Finished in 0.00094 seconds (files took 0.09055 seconds to load)
1 example, 0 failures
```

RSpec reruns the formerly failing example and verifies that it passes. If we try this process once more, RSpec won't have any failing examples left to run:

```
$ rspec --only-failures
Run options: include {:last_run_status=>"failed"}

All examples were filtered out

Finished in 0.00031 seconds (files took 0.08117 seconds to load)
0 examples, 0 failures
```

Another command-line option, --next-failure, offers a twist on this idea. You'll get a chance to try it out in the exercise at the end of this chapter.

Passing options to the rspec command isn't the only way to run just a subset of your examples. Sometimes, it's more convenient to make temporary annotations to your specs instead.

Focusing Specific Examples

If you find yourself running the same subset of specs repeatedly, you can save time by marking them as *focused*. To do so, simply add an f to the beginning of the RSpec method name:

- context becomes fcontext
- it becomes fit
- describe becomes fdescribe